

Executive Summary: ATC Readback Verifier

The safety problem

In air traffic control, a controller gives a pilot an instruction, a new altitude, a heading, a runway, a radio frequency, and the pilot reads it back so the controller can confirm it was heard correctly. This read-back is one of aviation's main defences against miscommunication, but it relies entirely on a busy human noticing a mistake in real time. When a read-back is wrong, incomplete, or has digits in the wrong order, the error can slip through. Wrong and missed read-backs are a documented contributor to aviation safety incidents, and research on reported cases suggests that a meaningful share of incorrect read-backs go uncaught. Spotting these slips quickly matters.

What we built

We built the **ATC Readback Verifier**, a simple web app that acts as an automatic second check on that exchange. A user types in the controller's instruction and the pilot's read-back. The tool reads both messages, breaks each one into the standard pieces of information an aircraft instruction contains, such as the callsign, altitude, heading, speed, runway, and radio frequency, and compares them side by side. It then returns a clear verdict: **MATCH** when the read-back is correct, or **DISCREPANCY** with a plain-language list of exactly what is wrong, for example a wrong altitude, a missing item, or a transposed runway number.

A deliberate design choice keeps the tool trustworthy: an AI language model is used only to pull the information out of the text. Every decision about what counts as correct is made by transparent, rule-based logic the team wrote, so the verdicts are predictable and can be inspected.

What we found

On our own hand-built test set of 50 example exchanges, covering correct read-backs and every type of error we set out to catch, the tool caught every error in the set and only rarely raised a false alarm, flagging a correct read-back as a problem in roughly 6% of cases. We also found that the way the AI model is prompted matters a great deal: an attempt to make the tool stricter by giving the model more examples actually made it worse, and the reliable fix came from tightening our own rule-based logic rather than the model. This confirmed that our comparison logic was sound and that reading the text correctly is the harder part.

The honest caveat

This is an early proof of concept, not operational software. It works on typed text only, it does not yet handle live audio or speech-to-text, and its accuracy depends on the AI model reading each message correctly, which is where its remaining mistakes come from. Live speech input is the clear next step.

Why it matters

The project shows that a lightweight, explainable tool can reliably check a pilot's read-back against the controller's instruction and explain its reasoning in plain terms. Because the judgement is rule-based rather than a black box, its decisions can be audited and trusted, a useful property for any aid that touches a safety-critical task, whether as a backstop, a review aid, or a training tool.